PROCESS SYNCHRONIZATION

Process Synchronization is the task of coordinating the execution of processes in a way that no two processes can have access to the same shared data and resources.

It is specially needed in a multi-process system when multiple processes are running together, and more than one processes try to gain access to the same shared resource or data at the same time.

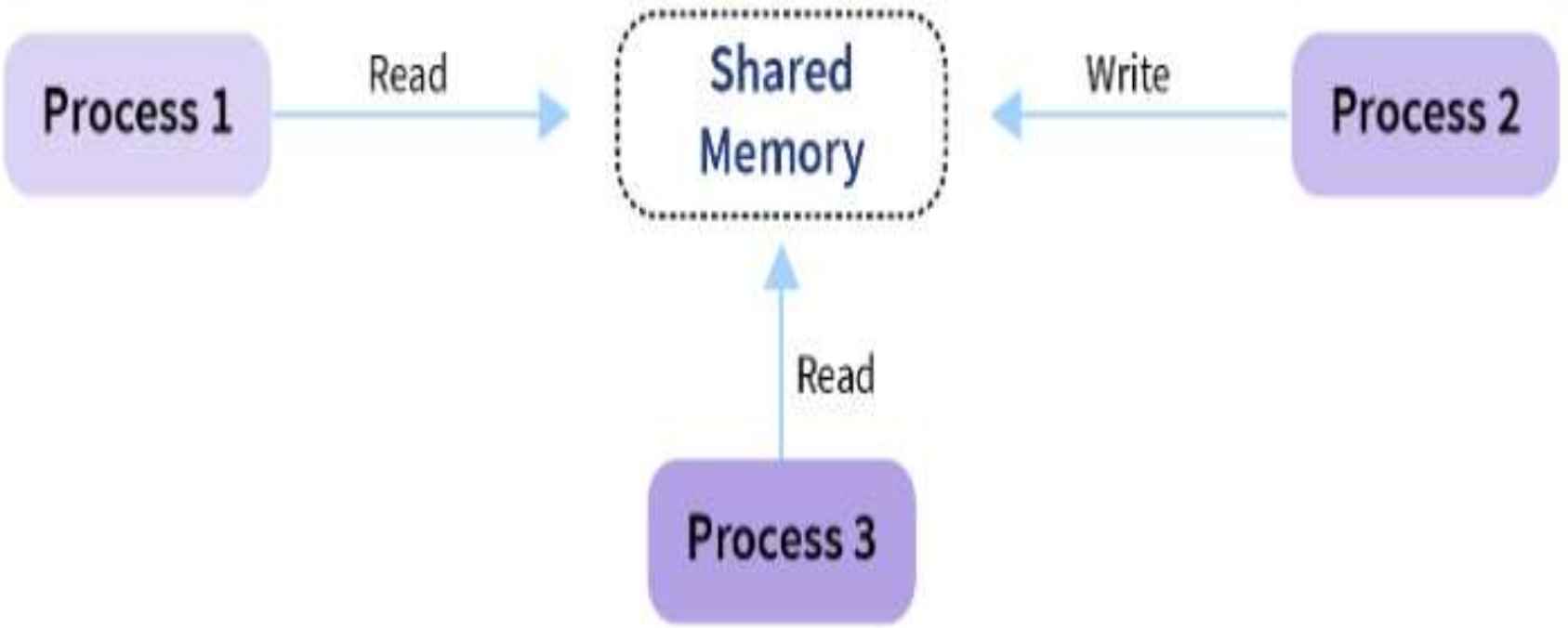
This can lead to the inconsistency of shared data. So the change made by one process is not necessarily reflected when other processes accessed the same shared data. To avoid this type of inconsistency of data, the processes need to be synchronized with each other.

On the basis of synchronization, processes are categorized as one of the following two types:

- **Independent Process** : Execution of one process does not affect the execution of other processes.
- **Cooperative Process** : Execution of one process affects the execution of other processes.

How Process Synchronization Works?

For Example, process 2 changing the data in a memory location while other processes 1 and 3 are trying to read the data from the **same** memory location. There is a high probability that data read by both these processes will be erroneous.



Process Synchronization is a procedure that is involved in preserving the appropriate order of execution of cooperative processes. In order to synchronize the processes, there are various synchronization mechanisms. Processes have to be scheduled to ensure that concurrent access to shared data does not create inconsistencies. Data inconsistency can result in what is called a race condition. A race condition occurs when two or more operations are executed at the same time, not scheduled in the proper sequence, and not exited in the critical section correctly.

Sections of a Program

- **Entry Section:** It is part of the process which decides the entry of a particular process.
- **Critical Section:** This part allows only one process to enter and modify the shared variable.
- **Exit Section:** Exit section allows the other process that are waiting in the Entry Section, to enter into the Critical Sections. It also checks that a process that finished its execution should be removed through this Section.
- **Remainder Section:** All other parts of the Code, which is not in Critical, Entry and Exit Section, are known as the Remainder Section.

- **Race Condition**

When more than one process is either running the same code or modifying the same memory or any shared data, there is a risk that the result or value of the shared data may be incorrect because all processes try to access and modify this shared resource. Thus, all the processes race to say that my result is correct. This condition is called the race condition. Since many processes use the same data, the results of the processes may depend on the order of their execution.

This is mostly a situation that can arise within the critical section. In the critical section, a race condition occurs when the end result of multiple thread executions varies depending on the sequence in which the threads execute.

But how to avoid this race condition? There is a simple solution:

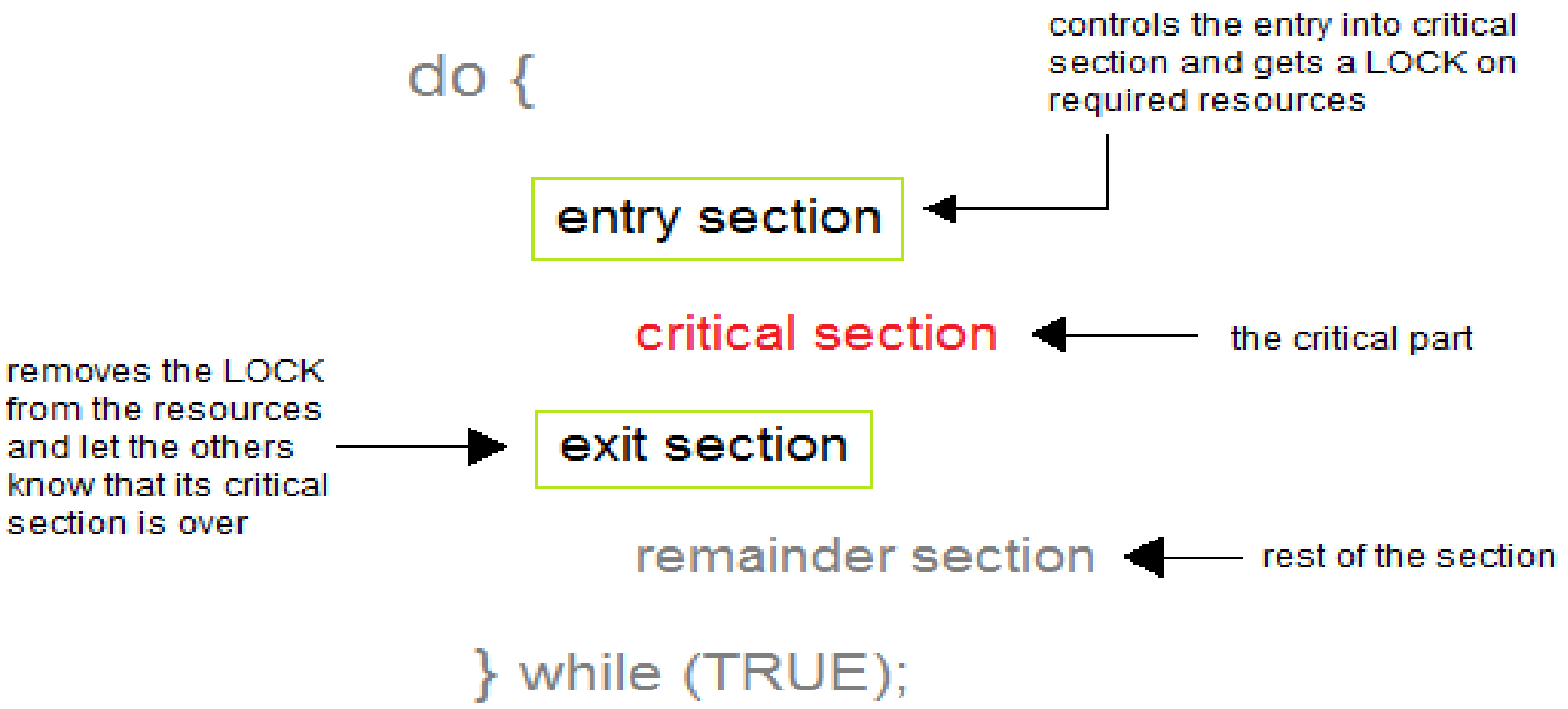
by treating the critical section as a section that can be accessed by only a single process at a time. This kind of section is called an atomic section.

- **Critical Section Problem**

A part of code that can only be accessed by a single process at any moment is known as a critical section. This means that when a lot of programs want to access and change a single shared data, only one process will be allowed to change at any given moment. The other processes have to wait until the data is free to be used.

The wait() function mainly handles the entry to the critical section, while the signal() function handles the exit from the critical section. If we remove the critical section, we cannot guarantee the consistency of the end outcome after all the processes finish executing simultaneously.

- The entry to the critical section is handled by the wait() function, and it is represented as P().
- The exit from a critical section is controlled by the signal() function, represented as V().



- Only one process can be executed inside the critical section at a time. Other processes waiting to execute their critical sections have to wait until the current process finishes executing its critical section.

Rules for Critical Section

The critical section need to must enforce all three rules:

- **Mutual Exclusion:** Our solution must provide mutual exclusion. By Mutual Exclusion, we mean that if one process is executing inside critical section then the other process must not enter in the critical section.
- **Progress:** Progress means that if one process doesn't need to execute into critical section then it should not stop other processes to get into the critical section.
- **Bound Waiting:** We should be able to predict the waiting time for every process to get into the critical section. The process must not be endlessly waiting for getting into the critical section.

- **Requirements of Synchronization**

The following three requirements must be met by a solution to the critical section problem:

- Mutual exclusion: If a process is running in the critical section, no other process should be allowed to run in that section at that time.
- Progress: If no process is still in the critical section and other processes are waiting outside the critical section to execute, then any one of the threads must be permitted to enter the critical section. The decision of which process will enter the critical section will be taken by only those processes that are not executing in the remaining section.
- No starvation: Starvation means a process that keeps waiting forever to access the critical section but never gets a chance. No starvation is also known as Bounded Waiting.
  - A process should not wait forever to enter inside the critical section.
  - When a process submits a request to access its critical section, there should be a limit or bound, which is the number of other processes that are allowed to access the critical section before it.
  - After this bond is reached, this process should be allowed to access the critical section.

Solutions for the Critical Section Problem

Some widely used method to solve the critical section problem are as follows:

Peterson's Solution

Peterson's approach to critical section problems is extensively utilized. It is a classical software-based solution.

The solution is based on the idea that when a process is executing in a critical section, then the other process executes the rest of the code and vice-versa is also possible, i.e., this solution makes sure that only one process executes the critical section at any point in time.

In Peterson's solution, we have two shared variables that are used by the processes.

- A boolean Flag[]: A boolean array Flag which is initialized to FALSE. This Flag array represents which process is which process wants to enter into the critical solution.
- int Turn: An integer variable Turn indicates the process number that is ready to enter into the critical section.

Flag array :



SCALER Topics

```
do{
    //A process Pi wants to enter into the critical section

    //The ith index of flag is set
    Flag[i] = True;
    Turn = i;
    while(Flag[i] && Turn == i);

    { Critical Section };

    Flag[i] = False;
    // another process can go to Critical Section
    Turn = j;

        Remainder Section

}  while ( True);
```

This solution preserves all three conditions:

- Mutual Exclusion is comforted as at any time only one process can access the critical section.
- Progress is also comforted, as a process that is outside the critical section does not block other processes from entering into the critical section.
- Bounded Waiting is assured as every process gets a fair chance to enter the Critical section.

Disadvantages of Peterson's Solution

- It involves busy waiting.(In the Peterson's solution, the code statement- "while(flag[j] && turn == j);" is responsible for this. Busy waiting is not favored because it wastes CPU cycles that could be used to perform other tasks). The repeated execution of a loop of code while waiting for an event to occur is called busy-waiting. The CPU is not engaged in any real productive activity during this period, and the process does not progress toward completion. Busy-waiting, busy-looping or spinning is a technique in which a process repeatedly checks to see if a condition is true, such as whether keyboard input or a lock is available. Busy waiting means a process simply spins, (does nothing but continue to test its entry condition) while it is waiting to enter its critical section.
- It is limited to 2 processes.
- Peterson's solution cannot be used in modern CPU architectures.

Spin lock

In software engineering, a spinlock is **a lock that causes a thread trying to acquire it to simply wait in a loop ("spin") while repeatedly checking whether the lock is available**. Since the thread remains active but is not performing a useful task, the use of such a lock is a kind of busy waiting.

Synchronization Hardware

Some times the problems of the Critical Section are also resolved by hardware. Some operating system offers a lock functionality where a Process acquires a lock when entering the Critical section and releases the lock after leaving it.

So when another process is trying to enter the critical section, it will not be able to enter as it is locked. It can only do so if it is free by acquiring the lock itself.

- **Mutex Locks**

As the synchronization hardware solution is not easy to implement for everyone, a strict software approach called Mutex Locks was introduced.

In this approach, in the entry section of code, a LOCK is acquired over the critical resources modified and used inside the critical section, and in the exit section that LOCK is released.

As the resource is locked while a process executes its critical section, no other process can access it.

•**TestAndSet**

TestAndSet is a hardware solution to the synchronization problem. In TestAndSet, we have a shared lock variable which can take either of the two values, 0 or 1.

- 0 Unlock
- 1 Lock

Before entering into the critical section, a process inquires about the lock. If it is locked, it keeps on waiting until it becomes free and if it is not locked, it takes the lock and executes the critical section.

In TestAndSet, Mutual exclusion and progress are preserved but bounded waiting cannot be preserved. As no queue is maintained for the processes stuck in the while loop, bounded waiting is not ensured. If a process waits for a set amount of time before entering the critical section, it is said to be a bounded waiting condition.

- **Semaphores**

A semaphore is a signaling mechanism, and a process can signal a process that is waiting on a semaphore. This differs from a mutex in that the mutex can only be notified by the process that sets the shared lock.

Semaphores make use of the wait() and signal() functions for synchronization among the processes.

A semaphore either allows or disallows access to the resource, which depends on how it is set up.

WAIT (S):

while (S <= 0);

S = S - 1;

SIGNAL (S):

S = S + 1;

The classical definitions of wait and signal are:

Wait:

This operation decrements the value of its argument S, as soon as it would become non-negative(greater than or equal to 1). This Operation mainly helps you to control the entry of a task into the critical section. In the case of the negative or zero value, no operation is executed. wait() operation was originally termed as P; so it is also known as P(S) operation. The definition of wait operation is as follows:

```
wait(S)
{
   while (S<=0);//no operation
   S--;                }
```

- **Signal:**

Increments the value of its argument S, as there is no more process blocked on the queue. This Operation is mainly used to control the exit of a task from the critical section.signal() operation was originally termed as V; so it is also known as V(S) operation.

```
signal(S)
{
S++;
}
```

Properties of Semaphores

- It's simple and always have a non-negative integer value.
- Works with many processes.
- Can have many different critical sections with different semaphores.
- Each critical section has unique access semaphores.
- Can permit multiple processes into the critical section at once, if desirable.

There are two types of semaphores:

- ***Counting Semaphores:***

Counting Semaphores can have any value and are not limited to a certain area. They can be used to restrict access to a resource that has a concurrent access limit.

Initially, the counting semaphores are set to the maximum amount of processes that can access the resource at a time. Hence, the counting semaphore indicates that a process can access the resource if it has a value greater than 0. If it is set to 0, no other process can access the resource. Hence,

When a process wants to use that resource, it first checks to see if the value of the counting semaphore is more than zero.

If yes, the process can then proceed to access the resource, which involves reducing the value of the counting semaphore by one.

When the process completes its critical section code, it can increase the value of the counting semaphore, making way for some other process to access it.

There are two types of semaphores:

- ***Binary Semaphores***

Binary Semaphores can only have one of two values: 0 or 1. Because of their capacity to ensure mutual exclusion, they are also known as mutex locks.

A single binary semaphore is shared between multiple processes.

When the semaphore is set to 1, it means some process is working on its critical section, and other processes need to wait, and if the semaphore is set to 0, that means any process can enter the critical section.

Hence, whenever the binary semaphore is set to 0, any process can then enter its critical section by setting the binary semaphore to 1. When it has completed its critical section, it can reset the binary semaphore to 0, enabling another process to enter it.

Advantages of Semaphores

- Semaphores allow only one process into the critical section. They follow the mutual exclusion principle strictly and are much more efficient than some other methods of synchronization.
- There is no resource wastage because of busy waiting in semaphores as processor time is not wasted unnecessarily to check if a condition is fulfilled to allow a process to access the critical section.
- Semaphores are implemented in the machine independent code of the microkernel. So they are machine independent.

Disadvantages of Semaphores

- Semaphores are complicated so the wait and signal operations must be implemented in the correct order to prevent deadlocks.
- Semaphores are impractical for last scale use as their use leads to loss of modularity. This happens because the wait and signal operations prevent the creation of a structured layout for the system.
- Semaphores may lead to a priority inversion where low priority processes may access the critical section first and high priority processes later.

The producer-consumer problem

Producer-Consumer problem is a classical synchronization problem in the operating system. With the presence of more than one process and limited resources in the system the synchronization problem arises. If one resource is shared between more than one process at the same time then it can lead to data inconsistency. In the producer-consumer problem, the producer produces an item and the consumer consumes the item produced by the producer.

- In operating System Producer is a process which is able to produce data/item.
- Consumer is a Process that is able to consume the data/item produced by the Producer.
- Both Producer and Consumer share a common memory buffer. This buffer is a space of a certain size in the memory of the system which is used for storage. The producer produces the data into the buffer and the consumer consumes the data from the buffer.

What's the problem?

- The producer should produce data only when the buffer is not full. If the buffer is full, then the producer shouldn't be allowed to put any data into the buffer.
- The consumer should consume data only when the buffer is not empty. If the buffer is empty, then the consumer shouldn't be allowed to take any data from the buffer.
- The producer and consumer should not access the buffer at the same time.

To solve this problem, we need two counting semaphores –

- **Full**

The full variable is used to track the space filled in the buffer by the Producer process. It is initialized to 0 initially as initially no space is filled by the Producer process.

- **Empty**

The Empty variable is used to track the empty space in the buffer. The Empty variable is initially initialized to the **BUFFER-SIZE** as initially, the whole buffer is empty.

- **Mutex**

Mutex is a binary semaphore variable that has a value of 0 or 1. It is used to achieve mutual exclusion. mutex ensures that at any particular time only the producer or the consumer is accessing the buffer.

Initialization of semaphores –

- **mutex = 1**
- **Full = 0 // Initially, all slots are empty. Thus full slots are 0**
- **Empty = n // All slots are empty initially**

We will use the Signal() and wait() operation in the above-mentioned semaphores to arrive at a solution to the Producer-Consumer problem.

- **Signal()** - The signal function increases the semaphore value by 1.
- **Wait()** - The wait operation decreases the semaphore value by 1.

Solution for Producer –

```
do{
    //produce an item
        wait(empty);
        wait(mutex);
        add();                  //place in buffer
        signal(mutex);
        signal(full);
} while(true)
```

When producer produces an item then the value of “empty” is reduced by 1 because one slot will be filled now. The value of mutex is also reduced to prevent consumer to access the buffer. Now, the producer has placed the item and thus the value of “full” is increased by 1. The value of mutex is also increased by 1 because the task of producer has been completed and consumer can access the buffer.

- wait(Empty) - Before producing items, the producer process checks for the empty space in the buffer. If the buffer is full producer process waits for the consumer process to consume items from the buffer. so, the producer process executes wait(Empty) before producing any item.
- wait(mutex) - Only one process can access the buffer at a time. So, once the producer process enters into the critical section of the code it decreases the value of mutex by executing wait(mutex) so that no other process can access the buffer at the same time.
- add() - This method adds the item to the buffer produced by the Producer process. once the Producer process reaches add function in the code, it is guaranteed that no other process will be able to access the shared buffer concurrently which helps in data consistency.
- signal(mutex) - Now, once the Producer process added the item into the buffer it increases the mutex value by 1 so that other processes which were in a busy-waiting state can access the critical section.
- signal(Full) - when the producer process adds an item into the buffer spaces is filled by one item so it increases the Full semaphore so that it indicates the filled spaces in the buffer correctly.

Solution for Consumer –

```
do{
wait(full);
wait(mutex);
consume();                    // remove item from buffer
signal(mutex);
signal(empty);
} while(true)
```

- wait(Full) - Before the consumer process starts consuming any item from the buffer it checks if the buffer is empty or has some item in it. So, the consumer process creates one more empty space in the buffer and this is indicated by the full variable. The value of the full variable decreases by one when the wait(Full) is executed. If the Full variable is already zero i.e the buffer is empty then the consumer process cannot consume any item from the buffer and it goes in the busy-waiting state.
- wait(mutex) - It does the same as explained in the producer process. It decreases the mutex by 1 and restricts another process to enter the critical section until the consumer process increases the value of mutex by 1.
- consume() - This function consumes an item from the buffer. when code reaches the consuming () function it will not allow any other process to access the critical section which maintains the data consistency.
- signal(mutex) - After consuming the item it increases the mutex value by 1 so that other processes which are in a busy-waiting state can access the critical section now.
- signal(Empty) - when a consumer process consumes an item it increases the value of the Empty variable indicating that the empty space in the buffer is increased by 1.

Readers-Writers Problem

Consider a situation where we have a file shared between many people.

If one of the people tries editing the file, no other person should be reading or writing at the same time, otherwise changes will not be visible to him/her.

If some person is reading the file, then others may read it at the same time.

In OS, this situation is known as the **readers-writers problem**.

Problem parameters:

- One set of data is shared among a number of processes
- Once a writer is ready, it performs its write. Only one writer may write at a time
- If a process is writing, no other process can read it
- If at least one reader is reading, no other process can write
- Readers may not write and only read

Solution:

The solution of readers and writers can be implemented using binary semaphores.

We use two binary semaphores "write" and "mutex",

Three variables are used: **mutex, wrt, readcnt** to implement solution

- **semaphore** mutex, wrt; // semaphore **mutex** is used to ensure mutual exclusion when **readcnt** is updated i.e. when any reader enters or exit from the critical section and semaphore **wrt** is used by both readers and writers
- **int** readcnt; // **readcnt** tells the number of processes performing read in the critical section, initially 0

Functions for semaphore :

- wait() : decrements the semaphore value.
- signal() : increments the semaphore value.

Writer process:

- Writer requests the entry to critical section.

- If allowed i.e. wait() gives a true value, it enters and performs the write. If not allowed, it keeps on waiting.
- It exits the critical section.

```
do {
    // writer requests for critical section
        wait(wrt);
    // performs the write
    // leaves the critical section
        signal(wrt);
} while(true);
```

Reader process:

- Reader requests the entry to critical section.
- If allowed:
  - it increments the count of number of readers inside the critical section. If this reader is the first reader entering, it locks the **wrt** semaphore to restrict the entry of writers if any reader is inside.

- It then, signals mutex as any other reader is allowed to enter while others are already reading.
- After performing reading, it exits the critical section. When exiting, it checks if no more reader is inside, it signals the semaphore “wrt” as now, writer can enter the critical section.

- If not allowed, it keeps on waiting.

```
do
{ // Reader wants to enter the critical section
Static int readcnt=0;
wait(mutex);
   // The number of readers has now increased by 1
       readcnt++;
   // there is atleast one reader in the critical section
   // this ensure no writer can enter if there is even one reader. Thus we give preference
      to readers here
     if (readcnt==1) {
     wait(wrt);    }
   // other readers can enter while this current reader is inside the critical section
     signal(mutex);
   // current reader performs reading here
```

```
wait(mutex);
// a reader wants to leave
readcnt--;
// that is, no reader is left in the critical section,
if (readcnt == 0)  {
signal(wrt);    }     // writers can enter
signal(mutex);     // reader leaves
} while(true);
```

Thus, the semaphore '**wrt**' is queued on both readers and writers in a manner such that preference is given to readers if writers are also there. Thus, no reader is waiting simply because a writer has requested to enter the critical section.

Writer Process:

```
wait(write);
WRITE INTO THE FILE
signal(wrt);
```

If a writer wishes to access the file, **wait** operation is performed on **write** semaphore, which decrements **write** to 0 and no other writer can access the file. On completion of the writing job by the writer who was accessing the file, the signal operation is performed on **write**.

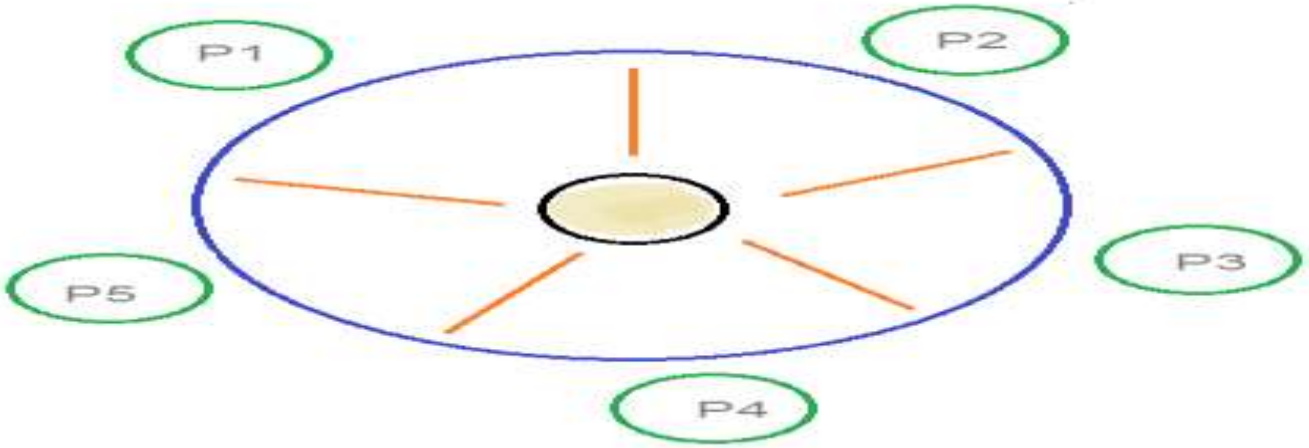
Dining Philosopher Problem

The dining philosophers problem is another classic synchronization problem which is used to evaluate situations where there is a need of allocating multiple resources to multiple processes.

Five philosophers are sitting around a circular table and their job is to think and eat alternatively. A bowl of noodles is placed at the center of the table along with five chopsticks for each of the philosophers. To eat a philosopher needs both their right and a left chopstick. A philosopher can only eat if both immediate left and right chopsticks of the philosopher is available. In case if both immediate left and right chopsticks of the philosopher are not available then the philosopher puts down their (either left or right) chopstick and starts thinking again.

Problem Statement:

Consider there are five philosophers sitting around a circular dining table. The dining table has five chopsticks and a bowl of rice in the middle as shown in the below figure.



At any instant, a philosopher is either eating or thinking. When a philosopher wants to eat, he uses two chopsticks - one from their left and one from their right. When a philosopher wants to think, he keeps down both chopsticks at their original place.

From the problem statement, it is clear that a philosopher can think for an indefinite amount of time. But when a philosopher starts eating, he has to stop at some point of time. The philosopher is in an endless cycle of thinking and eating.

SOLUTION

A solution of the Dining Philosophers Problem is to use a semaphore to represent a chopstick. A chopstick can be picked up by executing a wait operation on the semaphore and released by executing a signal semaphore.

The structure of the chopstick is shown below −

```
semaphore chopstick [5];
```

Initially the elements of the chopstick are initialized to 1 as the chopsticks are on the table and not picked up by a philosopher.

The code for each philosopher looks like:

```
do {
wait(chopstick[i]);
wait(chopstick[(i+1) % 5]); // mod is used because if i=5, next chopstick is 1
                            // (dining table is circular)
/* eat */
signal(chopstick[i]);
signal(chopstick[(i+1) % 5]);
/* think */
} while(1)
```

When a philosopher wants to eat the rice, he will wait for the chopstick at his left and picks up that chopstick. Then he waits for the right chopstick to be available, and then picks it too. After eating, he puts both the chopsticks down.

But if all five philosophers are hungry simultaneously, and each of them pickup one chopstick, then a deadlock situation occurs because they will be waiting for another chopstick forever. The possible solutions for this are:

- A philosopher must be allowed to pick up the chopsticks only if both the left and right chopsticks are available.
- Allow only four philosophers to sit at the table. That way, if all the four philosophers pick up four chopsticks, there will be one chopstick left on the table. So, one philosopher can start eating and eventually, two chopsticks will be available. In this way, deadlocks can be avoided.
- An even philosopher should pick the right chopstick and then the left chopstick while an odd philosopher should pick the left chopstick and then the right chopstick.

Monitors in Operating System

Monitors are used for process synchronization. With the help of programming languages, we can use a monitor to achieve mutual exclusion among the processes.

Example of monitors: ***Java Synchronized methods such as Java offers notify() and wait() constructs.***

In other words, monitors are defined as the construct of programming language, which helps in controlling shared data access.

The Monitor is a module or package which encapsulates shared data structure, procedures, and the synchronization between the concurrent procedure invocations.

Characteristics of Monitors.

- Inside the monitors, we can only execute one process at a time.
- Monitors are the group of procedures, and condition variables that are merged together in a special type of module.
- If the process is running outside the monitor, then it cannot access the monitor's internal variable. But a process can call the procedures of the monitor.

- Monitors offer high-level of synchronization
- Monitors were derived to simplify the complexity of synchronization problems.
- There is only one process that can be active at a time inside the monitor.

Components of Monitor

There are four main components of the monitor:

- **Initialization**: Initialization comprises the code, and when the monitors are created, we use this code exactly once.
- **Private data**: It comprises all the private data, and the private data contains private procedures that can only be used within the monitor. So, outside the monitor, private data is not visible.
- **Monitor procedure**: Monitors Procedures are those procedures that can be called from outside the monitor.
- **Monitor entry queue**: Monitor entry queue is another essential component of the monitor that includes all the threads, which are called procedures.

Syntax of monitor

```
Monitor Demo // Name of the Monitor
{
variables;
condition variables;
procedure p1 {.....}
procedure p2 {.....}
}
```

Condition Variables

There are two types of operations that we can perform on the condition variables of the monitor:

- Wait
- Signal

Suppose there are two condition variables

condition a, b *// Declaring variable*

Wait Operation

a.wait(): – The process that performs wait operation on the condition variables are suspended and locate the suspended process in a block queue of that condition variable.

Signal Operation

a.signal() : – If a signal operation is performed by the process on the condition variable, then a chance is provided to one of the blocked processes.

Advantages of Monitor

It makes the parallel programming easy, and if monitors are used, then there is less error-prone as compared to the semaphore.

Disadvantages of Monitor:

Monitors have to be implemented as part of the programming language . The compiler must generate code for them. This gives the compiler the additional burden of having to know what operating system facilities are available to control access to critical sections in concurrent processes. Some languages that do support monitors are Java,C#,Visual Basic,Ada and concurrent Euclid.